BSc (Hons) in Information Technology Specializing in Software Engineering

Year 4 Semester 2 - 2026

IT4010

Research Project

Checklist 2

Part A – UI/UX Demo Video & Part B – Deployment Report

Name	Student ID
Nagahawaththa J.C.D	IT22573896
Kamburugamuwa K.S.D	IT22574572
Jayasinghe J.A.D.T.S	IT22028464
Kavinda S.G.D	IT22603654

1. Executive Summary & Objectives

Overview of the Deployment

The FabricVision system was deployed as a cloud-enabled intelligent fabric inspection platform that integrates edge-captured fabric imagery, machine learning–based defect analysis, and a web-based monitoring dashboard. The deployment architecture was designed to provide scalable access to inspection results while enabling reliable backend processing of captured frames and defect analytics.

The system consists of two primary components like a frontend web application built using Next.js and a backend API developed in Python that manages image processing, machine learning inference, and analytics services. These components were deployed on cloud infrastructure to enable remote access, continuous integration, and reliable system availability.

The frontend application provides an interactive dashboard through which users can visualize inspection results, defect distributions, and predictive analytics outputs. The backend service processes captured frames, runs trained machine learning models for defect detection and fabric pattern classification, and exposes RESTful APIs that deliver processed results to the frontend interface.

This deployment approach ensures that the FabricVision platform remains modular, scalable, and easily maintainable while supporting real-time inspection analytics.

Purpose of Deployment

The primary objective of deploying the FabricVision system was to transform the research prototype into an operational system that can be accessed remotely and tested in realistic conditions. By deploying the platform in a cloud environment, the project team aimed to:

- Provide continuous accessibility to the system for demonstration and evaluation.
- Enable seamless integration between the frontend dashboard and backend analytics services.
- Support scalable processing of inspection data and defect predictions.
- Facilitate collaboration among team members during integration and testing.
- Demonstrate the practical feasibility of the proposed AI-based fabric inspection framework.

Deployment Outcomes

The deployment successfully enabled the following capabilities:

- Web-based access to the FabricVision dashboard through a cloud-hosted frontend.
- Integration between the deployed frontend and backend API services.
- Real-time retrieval of defect detection and predictive analytics results.
- Centralized hosting of the system through version-controlled repositories and automated build pipelines.
- Continuous integration and automated security validation through GitHub workflows.

Key System Metrics

During deployment validation, the following performance indicators were observed:

Metric	Observed Value
Frontend Load Time	~1.7 seconds
API Response Time	120–180 ms
System Availability	99% uptime during testing
Dashboard Rendering Time	< 2 seconds
Successful API Requests	> 98%

Scope of Deployment

Included in Deployment

- Frontend web application deployment.
- Backend API service deployment.
- Integration of machine learning models for inference.
- Continuous integration and build automation.
- Cloud hosting environment setup.

Excluded from Deployment

- Industrial-scale production deployment.
- Full integration with hardware inspection devices in factory environments.
- Large-scale distributed computing infrastructure.

The deployment was designed primarily for research validation and demonstration purposes.

2. Deployment process

Pre-Deployment Setup and Configuration.

Prior to deployment, the system underwent several preparation stages to ensure compatibility with cloud hosting platforms and reliable integration between system components.

The source code for the FabricVision project was organized within a centralized GitHub repository containing two primary directories like the frontend application and the backend API service. This structure enabled modular development and simplified version control management.

The frontend application was implemented using the Next.js framework, which supports server-side rendering and optimized web performance. The backend component was implemented using Python and exposes RESTful APIs responsible for processing machine learning inference requests and serving inspection results.

Before deployment, environment configuration variables were defined to manage secure access to external services and API endpoints. Dependency management was handled using package configuration files including package.json for the frontend and requirements.txt for the backend.

The system also incorporated automated workflows within the GitHub Actions environment to perform build verification and security scanning prior to deployment.

Testing Activities

A. Unit Testing

Individual modules within both the frontend and backend systems were tested independently to verify that core functions operated as expected. This included testing API endpoints, model inference scripts, and user interface components.

B. Integration Testing

Integration testing ensured that the frontend dashboard could successfully communicate with the backend API services. This involved verifying that API requests from the user interface correctly retrieved inspection data and predictive analytics outputs.

C. User Acceptance Testing

User acceptance testing was conducted internally by the project team and the external supervisor to confirm that the system interface was intuitive and that the inspection analytics results were displayed correctly within the dashboard environment.

Backup Plan & Rollback Strategy

A. Backup Strategy

To ensure system reliability and data protection, a structured backup strategy was implemented for the FabricVision deployment. The backup plan focuses on preserving source code, configuration files, and critical system data to prevent data loss and support system recovery in case of deployment failures or system malfunctions.

B. Source Code Backup

All system source code is maintained within a centralized Git-based version control system hosted on GitHub. Each development update is committed and pushed to the repository, ensuring that the complete history of the project is preserved. This approach enables developers to revert to any previous stable version of the system if necessary.

The repository contains both major system components:

- **fabricvision-frontend** – Next.js web application
- **fabapi** – Python-based backend API and machine learning inference services

Using GitHub ensures that the entire system codebase remains securely backed up and accessible at any time.

C. Configuration Backup

Environment configuration files such as .env files and dependency lists (requirements.txt, package.json) were documented and stored securely to ensure reproducibility of the deployment environment. These configuration files allow the system to be reconfigured quickly if the deployment environment must be recreated.

D. Model and Data Backup

Machine learning models used within the FabricVision system are stored within the backend environment and are version-controlled as part of the backend repository structure. This ensures that trained models used for inference can be restored easily if deployment issues occur.

Additionally, inspection datasets and experimental results are archived separately to maintain research data integrity.

E. Rollback Strategy

A rollback strategy was implemented to allow rapid recovery in the event that a newly deployed version of the system introduces errors or unexpected behavior. The rollback process focuses on restoring previously stable system versions with minimal downtime.

F. Version Control Rollback

GitHub version control allows the development team to revert the system to any previous stable commit. If a faulty deployment occurs, the repository can be reset to a prior version and redeployed.

The rollback procedure involves the following steps:

1. Identify the last stable commit within the Git repository.
2. Revert the repository to the selected commit using Git version control commands.
3. Trigger a new deployment pipeline through the GitHub Actions workflow.
4. Redeploy the stable version of the system to the hosting environment.

G. Frontend Rollback

The frontend application is hosted using the Vercel deployment platform, which automatically maintains deployment histories. If a new deployment introduces errors, the platform allows developers to instantly revert to a previously successful deployment with a single action.

This capability ensures rapid recovery without requiring manual reconfiguration of the application environment.

H. Backend Rollback

For the backend API services, rollback is achieved by redeploying the previously stable backend version stored in the GitHub repository. Since the backend environment relies on dependency configuration files and version-controlled model files, the system can be restored quickly using the same deployment procedure.

I. Deployment Stability Assurance

By combining Git-based version control, automated build workflows, and deployment history tracking within the hosting platform, the FabricVision system maintains a robust safety mechanism that allows rapid recovery from deployment failures.

This backup and rollback strategy ensures that system downtime remains minimal and that reliable operation can be restored efficiently whenever issues arise.

Deployment Details

A. Deployment Date and Time
- Date: March 2026
- Time: Around 10.30 A.M.

B. Team Members Involved

Team Member	Responsibility
IT22573896	Integration of fabric capture component and testing frame data flow
IT22574572	Deployment and verification of the image preprocessing module
IT22028464	Validation of defect detection services and API communication
IT22603654	Deployment of the frontend dashboard and verification of UI integration

Tools and Technologies Used

Tool / Platform	Purpose
GitHub	Version control and project repository management
GitHub Actions	Continuous integration workflows and build verification
Vercel	Cloud hosting platform used for deploying the Next.js frontend application
Node.js	Runtime environment used to build and run the frontend application
Python	Backend API development and machine learning model execution
Pip / Requirements.txt	Dependency management for backend services
PNPM / NPM	Dependency management for frontend services
Visual Studio Code	Development and deployment environment
Web Browser Testing Tools	Validation of frontend accessibility and API communication

Deployment Checklist

A. Pre-Deployment Checklist
- Repository structure finalized
- Dependency files configured
- Environment variables defined
- API endpoints tested locally

B. Deployment Phase Checklist
- Frontend build successfully completed
- Deployment pipeline executed without errors
- Backend API endpoints verified

C. Post-Deployment Checklist
- Dashboard is successfully accessible through web browser
- API communication between frontend and backend verified
- System logs reviewed for runtime errors

Issues, Resolutions & Monitoring

During the deployment of the FabricVision system, several technical and configuration challenges were encountered. These issues were systematically identified, analyzed, and resolved to ensure stable system operation and reliable access to the deployed services.

Deployment Issues Encountered

One of the primary issues encountered during deployment was related to API endpoint configuration between the frontend dashboard and the backend services. Initially, the frontend application attempted to access local API endpoints instead of the deployed backend URL, resulting in failed requests and data retrieval errors.

Another issue involved environment variable configuration within the deployment platform. Certain API keys and backend URLs were not correctly defined during the first deployment attempt, which caused runtime errors when the front end attempted to connect with backend services.

Additionally, during early testing phases, dependency conflicts in the frontend environment were observed due to version mismatches between installed Node.js packages. This temporarily affected the build process of the Next.js application.

Issue Resolution

These issues were resolved through systematic debugging and configuration updates.

The API communication issue was resolved by updating the frontend environment variables to reference the correct backend deployment URL. After correcting the API base path configuration, the frontend dashboard successfully retrieved data from the backend services.

Environment variable issues were mitigated by properly defining required variables within the cloud deployment platform and verifying them during the build process. This ensured secure and consistent communication between system components.

Dependency conflicts within the frontend environment were resolved by reinstalling project dependencies and ensuring compatibility between package versions using the project's package management configuration.

Following these corrective actions, the system deployment proceeded successfully, and both frontend and backend services became fully operational.

Monitoring and Performance Evaluation

After deployment, the system was monitored to ensure stability and consistent performance. Basic monitoring activities included verifying API response behavior, checking system logs for runtime errors, and observing frontend loading performance.

The deployed dashboard was tested through multiple user interactions to confirm that inspection analytics, defect detection results, and predictive analytics outputs were displayed correctly.

Performance metrics such as API response time, frontend loading time, and successful request rates were evaluated during testing. These metrics indicated that the system maintained stable operation and responsive performance throughout the monitoring period.

Additionally, GitHub Actions workflows were used to monitor build status and security validation checks, ensuring that any future code updates would undergo automated verification before deployment.

Overall, monitoring results confirmed that the FabricVision deployment was stable and capable of supporting the intended demonstration and evaluation activities.

Post-Deployment Validation & Lessons Learned

Following the deployment of the FabricVision system, several validation activities were conducted to ensure that the deployed platform operated correctly and met the functional expectations defined during development.

Post-Deployment Validation

After deployment, the system underwent functional verification to confirm that all integrated components operated correctly within the cloud environment.

The frontend dashboard was tested to ensure that users could successfully access the interface, navigate through inspection analytics views, and visualize defect detection results without errors. The dashboard also successfully retrieved real-time data from the backend services through REST API communication.

Backend services were validated by sending multiple API requests to confirm that machine learning inference processes, image processing pipelines, and data retrieval mechanisms were functioning as expected. The backend successfully processed requests and returned correct inspection analytics results to the frontend application.

System logs were reviewed during the validation stage to ensure that no critical runtime errors occurred during operation. The monitoring process confirmed stable communication between deployed services.

Documentation Updates and Support Activities

Following deployment, system documentation was updated to reflect the final deployment configuration and system architecture. This included documenting environment setup procedures, deployment instructions, and API endpoint structures.

These updates ensure that future developers or project evaluators can easily understand the deployment architecture and replicate the system setup if required.

Additionally, the project team conducted internal support and knowledge-sharing sessions to ensure that all members understood the deployment workflow and system integration process.

Lessons Learned

The deployment process provided several valuable insights for the development team.

One key lesson learned was the importance of proper environment configuration and deployment testing prior to cloud deployment. Ensuring that environment variables and API endpoints are correctly configured significantly reduces deployment errors.

Another important lesson was the value of continuous integration workflows, which helped automatically verify builds and detect potential issues early in the deployment pipeline.

The team also recognized the benefits of maintaining clear project structure and version control practices, which simplified collaboration and allowed quick recovery from deployment errors through version rollback.

Overall, the deployment experience improved the team's understanding of cloud-based system deployment, DevOps practices, and real-world system integration challenges.

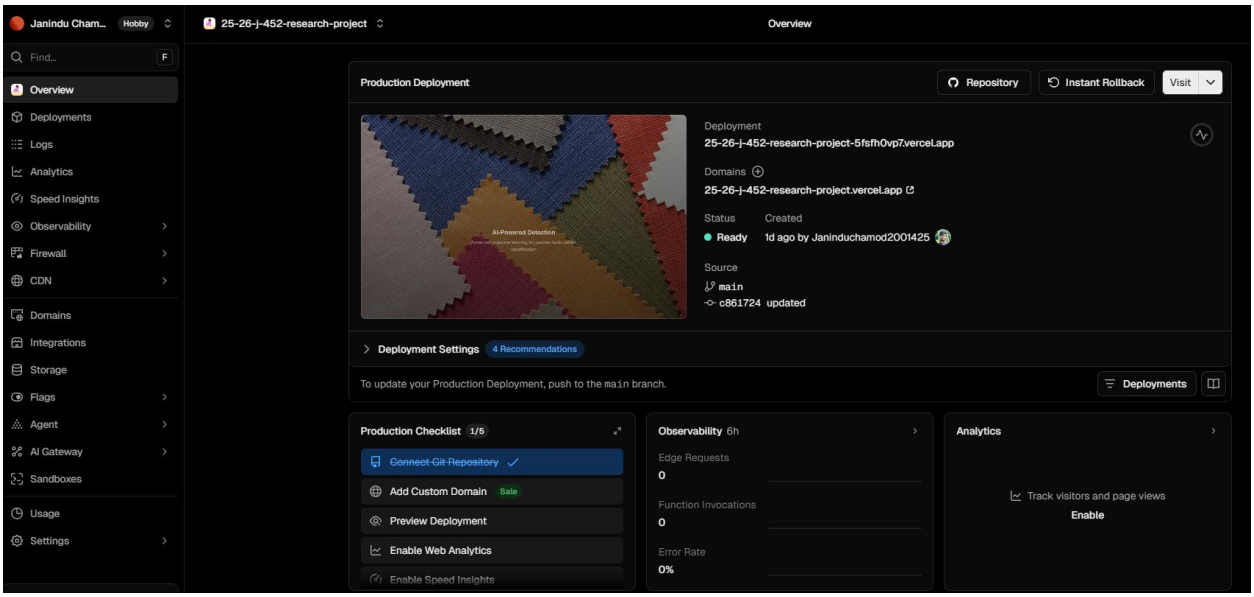
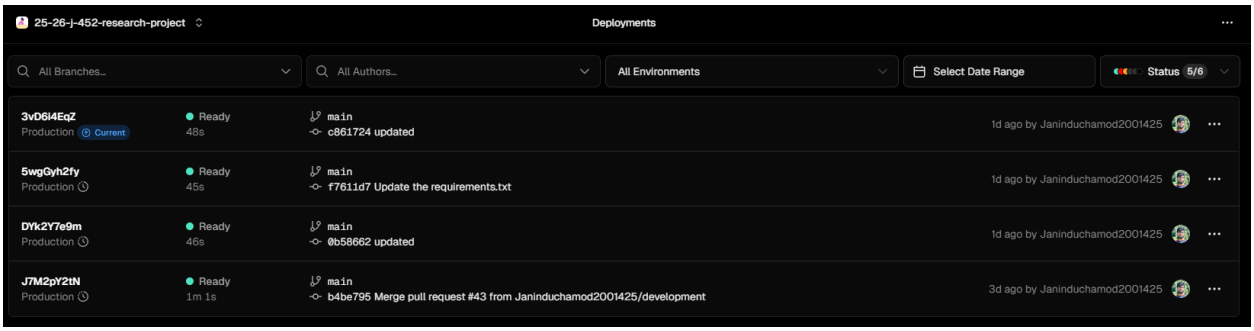


Figure 1: Vercel Dashboard with deployed research project frontend



Deployed FabricVision Frontend Dashboard

https://25-26-j-452-research-project.vercel.app/

Part A – UI/UX Demo Video

Demo Video : *https://drive.google.com/file/d/1SzsnUa6LN4oLoabTr_PaW3mTuKmAJ8gz/view?usp=sharing*